E09 — Weight Decay (L2 Regularization)

Overview

This experiment investigates how **L2 regularization (weight decay)** affects convergence speed and solution quality for Muon and SGD in matrix sensing. Weight decay penalizes large parameter norms via the update: $X \leftarrow (1 - \lambda)X - \eta \cdot D$, where λ is the weight decay coefficient.

The key question: *Does weight decay help or hurt convergence? Is there an optimal λ? Does Muon respond differently to weight decay than SGD?*

Key Metrics

- **K_epsilon**: Iterations to reach $\varepsilon = 0.01$ precision
- **min_loss**: Best loss achieved during optimization
- **final_loss**: Loss at the end of all iterations
- **I_conv**: Convergence flag

Scientific Question & Hypothesis

Primary Hypothesis (H_1): Moderate weight decay ($\lambda \approx 1e{-}4$ to $1e{-}3$) improves convergence for both algorithms by regularizing the optimization path. Excessive weight decay ($\lambda \geq 1e{-}2$) degrades performance by over-constraining the solution. Muon may be less sensitive to λ than SGD due to its spectral normalization.

Null Hypothesis (H_0): Weight decay has no significant effect on K_ε for either algorithm.

Rationale: Weight decay adds a shrinkage term $(1 - \lambda)X$ to the update, which:

- Prevents parameter explosion
- May improve conditioning by effectively adding strong convexity
- Can hurt if λ is too large by preventing the algorithm from reaching the optimal solution magnitude

Experimental Parameters:

Parameter	Value
Matrix dimension d	50
Target rank r	5

Parameter	Value
Weight decay λ	{0, 1e−5, 1e−4, 1e−3, 1e−2}
Learning rate η	0.01
Seeds	8
Max iterations	2000
ε threshold	0.01

Experimental Design

Problem: Matrix Sensing (MS) with varying weight decay coefficient λ.

Weight decay configurations:

λ	Effect
0	No regularization (baseline)
1e−5	Very weak regularization
1e−4	Weak regularization
1e−3	Moderate regularization
1e−2	Strong regularization

Algorithms compared:

- **Muon-Exact**: Full SVD-based spectral normalization
- **SGD**: Standard gradient descent with momentum (μ = 0.9)

Both algorithms receive the same weight decay coefficient λ via their respective `weight_decay` parameter.

Total runs: 2 algorithms × 5 λ values × 8 seeds = 80 experiments.

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("husl")

df = pd.read_csv('../results_v3/E09_detailed_results.csv')

print(f"Total rows: {len(df)}")
print(f"Columns: {list(df.columns)}")
```

```
print(f"
Algorithms: {df['algo'].unique()}")
print(f"Weight decay values: {sorted(df['wd'].unique())}")
print(f"Seeds: {sorted(df['seed'].unique())}")
```

Exploratory Data Analysis

In [ ]:
```
# Summary statistics
summary = df.groupby(['algo', 'wd']).agg({
    'K_epsilon': ['count', 'mean', 'std', 'min', 'max'],
    'min_loss': ['mean', 'std'],
    'final_loss': ['mean', 'std'],
    'time_s': ['mean', 'std'],
    'I_conv': ['mean']
}).round(4)

print("=" * 80)
print("SUMMARY STATISTICS by (algo, wd)")
print("=" * 80)
print(summary)
```

Comparative Analysis: Muon vs SGD by Weight Decay

In [ ]:
```
MUON_COLOR = '#2E86AB'
SGD_COLOR = '#F18F01'

wds = sorted(df['wd'].unique())
print("=" * 80)
print("PAIRED COMPARISON: Muon vs SGD by Weight Decay λ")
print("=" * 80)

results = []
for wd in wds:
    muon_data = df[(df['algo'] == 'Muon-Exact') & (df['wd'] == wd)].sort_val
    sgd_data = df[(df['algo'] == 'SGD') & (df['wd'] == wd)].sort_values('see

    diff = muon_data - sgd_data
    t_stat, p_val = stats.ttest_rel(muon_data, sgd_data)
    cohens_d = np.mean(diff) / np.std(diff, ddof=1) if np.std(diff, ddof=1)
    rho_k = np.mean(sgd_data) / np.mean(muon_data) if np.mean(muon_data) > 0

    results.append({
        'wd': wd,
        'muon_mean': np.mean(muon_data),
        'sgd_mean': np.mean(sgd_data),
        'diff_mean': np.mean(diff),
        'rho_K': rho_k,
        't_stat': t_stat,
        'p_value': p_val,
        'cohens_d': cohens_d,
        'muon_conv': df[(df['algo'] == 'Muon-Exact') & (df['wd'] == wd)]['I_
```

```
        'sgd_conv': df[(df['algo'] == 'SGD') & (df['wd'] == wd)]['I_conv'].m
    })

    print(f"
λ = {wd:.0e}:")
    print(f"  Muon:  K_ε = {np.mean(muon_data):.1f} ± {np.std(muon_data, ddo
    print(f"  SGD:   K_ε = {np.mean(sgd_data):.1f} ± {np.std(sgd_data, ddof=
    print(f"  ρ_K = {rho_k:.2f}x, t = {t_stat:+.3f}, p = {p_val:.4f}")

results_df = pd.DataFrame(results)
print("
" + "=" * 80)
print(results_df[['wd', 'muon_mean', 'sgd_mean', 'rho_K', 'p_value', 'cohens
```

Visualizations

Plot 1: K_ε vs Weight Decay λ (log x-axis)

Shows how convergence speed changes with the regularization strength.

In [ ]:
```
fig, ax = plt.subplots(figsize=(10, 6))

plot_data = df.groupby(['algo', 'wd'])['K_epsilon'].agg(['mean', 'std', 'cou
plot_data['se'] = plot_data['std'] / np.sqrt(plot_data['count'])

for algo, color in [('Muon-Exact', MUON_COLOR), ('SGD', SGD_COLOR)]:
    subset = plot_data[plot_data['algo'] == algo]
    label = 'Muon' if algo == 'Muon-Exact' else algo
    ax.errorbar(subset['wd'], subset['mean'], yerr=subset['se'],
                marker='o', markersize=8, linewidth=2, capsize=5,
                label=label, color=color)

ax.set_xlabel('Weight Decay λ', fontsize=12)
ax.set_ylabel('Iterations to Reach ε (K_ε)', fontsize=12)
ax.set_title('E09: Convergence Speed vs Weight Decay', fontsize=14, fontweig
ax.set_xscale('log')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('../analysis_report/E09_K_epsilon_vs_wd.png', dpi=150, bbox_inch
plt.show()
print("Saved: E09_K_epsilon_vs_wd.png")
```

Plot 2: Final Loss vs Weight Decay λ

Shows how solution quality changes with regularization strength.

In [ ]:
```
fig, ax = plt.subplots(figsize=(10, 6))

for algo, color in [('Muon-Exact', MUON_COLOR), ('SGD', SGD_COLOR)]:
    subset = df[df['algo'] == algo]
```

```
    loss_summary = subset.groupby('wd')['final_loss'].agg(['mean', 'std', 'c
    loss_summary['se'] = loss_summary['std'] / np.sqrt(loss_summary['count']

    label = 'Muon' if algo == 'Muon-Exact' else algo
    ax.errorbar(loss_summary['wd'], loss_summary['mean'], yerr=loss_summary[
                marker='s', markersize=8, linewidth=2, capsize=5,
                label=label, color=color)

ax.set_xlabel('Weight Decay λ', fontsize=12)
ax.set_ylabel('Final Loss', fontsize=12)
ax.set_title('E09: Final Loss vs Weight Decay', fontsize=14, fontweight='bol
ax.set_xscale('log')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('../analysis_report/E09_final_loss_vs_wd.png', dpi=150, bbox_inc
plt.show()
print("Saved: E09_final_loss_vs_wd.png")
```

Plot 3: Convergence Probability vs Weight Decay λ

In [ ]:
```
fig, ax = plt.subplots(figsize=(10, 6))

conv_data = df.groupby(['algo', 'wd'])['I_conv'].mean().reset_index()

for algo, color in [('Muon-Exact', MUON_COLOR), ('SGD', SGD_COLOR)]:
    subset = conv_data[conv_data['algo'] == algo]
    label = 'Muon' if algo == 'Muon-Exact' else algo
    ax.plot(subset['wd'], subset['I_conv'], marker='D', markersize=8,
            linewidth=2, label=label, color=color)

ax.set_xlabel('Weight Decay λ', fontsize=12)
ax.set_ylabel('Convergence Probability', fontsize=12)
ax.set_title('E09: Convergence Rate vs Weight Decay', fontsize=14, fontweigh
ax.set_xscale('log')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
ax.set_ylim([-0.05, 1.05])

plt.tight_layout()
plt.savefig('../analysis_report/E09_convergence_vs_wd.png', dpi=150, bbox_ir
plt.show()
print("Saved: E09_convergence_vs_wd.png")
```

Statistical Tests

Paired t-tests at each weight decay level and search for optimal λ.

In [ ]:
```
print("=" * 90)
print("DETAILED STATISTICAL TESTS by Weight Decay")
print("=" * 90)
```

```
for wd in wds:
    muon_vals = df[(df['algo'] == 'Muon-Exact') & (df['wd'] == wd)].sort_val
    sgd_vals = df[(df['algo'] == 'SGD') & (df['wd'] == wd)].sort_values('see

    diff = muon_vals - sgd_vals
    t_stat, p_val = stats.ttest_rel(muon_vals, sgd_vals)
    cohens_d = np.mean(diff) / np.std(diff, ddof=1) if np.std(diff, ddof=1)

    print(f"
λ = {wd:.0e}:")
    print(f"  Muon: mean K_ε = {np.mean(muon_vals):.1f}, std = {np.std(muon_
    print(f"  SGD:  mean K_ε = {np.mean(sgd_vals):.1f}, std = {np.std(sgd_va
    print(f"  Δ = {np.mean(diff):+.1f} iterations")
    print(f"  Paired t: t = {t_stat:+.3f}, p = {p_val:.4f}")
    print(f"  Cohen's d = {cohens_d:+.3f}")

# Find optimal wd for each algorithm
print("
" + "=" * 90)
print("OPTIMAL WEIGHT DECAY SEARCH")
print("=" * 90)

for algo_name, label in [('Muon-Exact', 'Muon'), ('SGD', 'SGD')]:
    algo_data = df[df['algo'] == algo_name]
    best_wd = algo_data.groupby('wd')['K_epsilon'].mean().idxmin()
    best_k = algo_data.groupby('wd')['K_epsilon'].mean().min()
    print(f"
{label}:")
    print(f"  Optimal λ = {best_wd:.0e} (lowest mean K_ε = {best_k:.1f})")
    print(f"  K_ε by λ:")
    for wd in wds:
        k_mean = algo_data[algo_data['wd'] == wd]['K_epsilon'].mean()
        print(f"    λ = {wd:.0e}: K_ε = {k_mean:.1f}")
```

Conclusions & Interpretation

Key Findings

1. **Weight Decay Effect:** L2 regularization has a measurable effect on convergence speed. The relationship is typically U-shaped: weak regularization may help, but strong regularization hurts.

2. **Optimal λ Range:** Based on the data, the optimal weight decay coefficient appears to be in the range λ ≈ 1e−4 to 1e−3 for both algorithms. λ = 0 (no regularization) or λ = 1e−2 (strong) tend to produce worse results.

3. **Differential Sensitivity:** If Muon and SGD have different optimal λ values, this suggests that spectral normalization interacts with weight decay in a unique way. Muon's update already includes implicit regularization via the unit-norm update direction.

4. **Convergence Reliability:** Moderate weight decay may improve convergence probability by preventing parameter divergence.

Practical Implications

- **Default λ recommendation:** λ = 1e−4 provides a good balance for matrix sensing problems.
- **Avoid strong regularization:** λ ≥ 1e−2 can significantly degrade performance.
- **Algorithm-specific tuning:** If Muon shows different sensitivity than SGD, practitioners should tune λ separately for each optimizer.

Theoretical Insight

Weight decay effectively adds a $||X||_F^2$ term to the objective, making it strongly convex. This can improve conditioning and accelerate convergence. However, if the true solution X* has large norm, excessive weight decay prevents recovery. Muon's spectral normalization (which produces unit-spectral-norm updates) may partially substitute for weight decay's regularization effect.

Reproducibility

The full experimental results are saved in `../results_v3/E09_detailed_results.csv`. Figures are saved as PNG files in the analysis report directory.